A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Thai Nguyen Vu[1] Dung Van Bui[2] Nhut Tri Do[2] Van Du Nguyen[3]

SOICT 2026

[1]University of Transport and Communications, HCMC Campus [2]University of Information Technology, VNU-HCM
[3]Nong Lam University, HCMC

I. PROBLEM AND CONTRIBUTIONS

Edge intrusion detection: three open gaps

Detectors now *fit* on single-board hardware — but a monolithic one classifies **every** flow and saturates an 8 GB ARM board.

What the literature does not tell us

1. Everything is measured on a **single device**.
2. Throughput, latency and **energy** are never reported together.
3. Offline scores come from **leaky protocols**.

Our question

How should inference be *placed* across a constrained two-node edge cluster?

Our answer

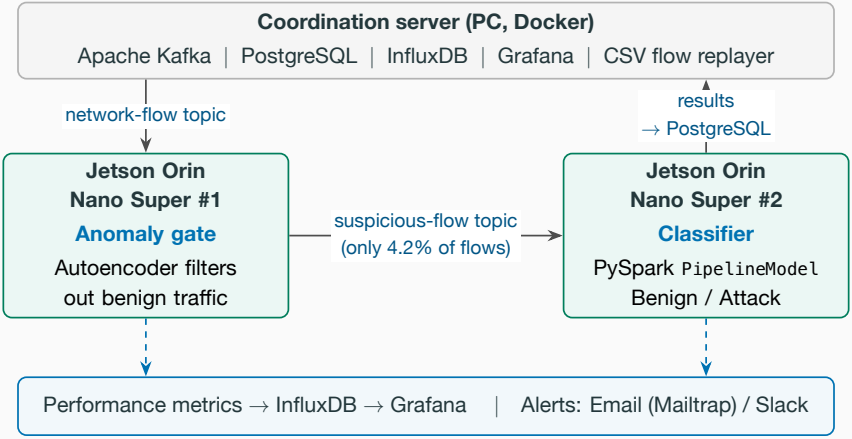
Split it: a cheap **autoencoder gate** on board #1, the heavy **PySpark classifier** on board #2, joined by a Kafka topic.

Contributions

1. A **two-stage distributed IDS**: anomaly gate + Spark classifier, physically separated across two boards.
2. **Three deployment modes** for a two-node Jetson cluster — pipeline split, horizontal Kafka scaling, Spark standalone cluster.
3. An **end-to-end evaluation**: throughput, latency percentiles, per-node CPU/RAM *and* energy per verdict.
4. **Leakage-aware provenance** — the deployed model is exactly the one our companion study selected under a leakage-free protocol.

II. SYSTEM AND EVALUATION SETUP

Split-deployment architecture



The gate's MSE threshold is calibrated *offline* on a held-out benign validation split — committed before evaluation, never tuned on the test set.

Three deployment modes, one workload

Single node full pipeline in one process — the reference.

A: Pipeline split gate on #1, classifier on #2 (our design).

B: Horizontal both boards run the *full* role, sharing a Kafka consumer group.

C: Spark cluster host as master, both boards as workers.

Throughput is counted as **final verdicts per second** — a flow is decided at the gate *or* at the classifier — so forwarded flows are never double-counted.

Setup

2× Jetson Orin Nano Super (6-core A78AE, 8 GB, ≈67 TOPS)

PySpark 3.4.1, Kafka 3.5, Wi-Fi LAN

CICIDS2017 replay at 100 flows/s

≥3 repetitions, warmup excluded

Energy via `tegrastats`, idle subtracted

III. RESULTS AND CONCLUSION

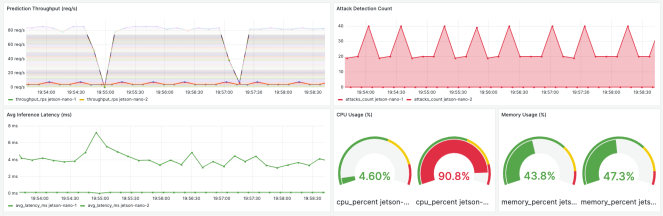
Result 1: splitting the pipeline buys 24× throughput

Mode	Throughput (flows/s)	p95 latency (ms)	Gate skip (%)	Energy/verdict (mJ)
Single node	2.60 ± 0.41	12.3 ± 3.0	(inline)	2490
A: Pipeline split	**62.5 ± 0.6**	**13.1 ± 2.6**	**95.8**	**178**
B: Horizontal	5.9 ± 1.4	21.6 ± 19.2	(inline)	2250
C: Spark cluster	90.0 ± 25.6	23.9 ± 17.8	97.9	119

- **A: 24× throughput at unchanged p95** — the gate absorbs 95.8% of benign traffic.
- **B stays Spark-bound** (2×): every node still classifies.
- C is fastest on average but ±26 flows/s and needs the host master.
- Energy amortised over 24–35× more verdicts: **2.5 → 0.12–0.18 J**.

Pipeline split is the recommended default — the highest throughput *per unit of variance*; cluster mode is for models that exceed single-node memory.

Result 2: the load asymmetry is visible live



At peak load, split mode:

- gate node: **4.6% CPU**
- classifier node: **90.8% CPU**

The classifier dominates the compute budget — which is exactly why moving it off the ingest path pays.

Result 3: the honest price of one unified stack

We serve with Spark for *consistency with distributed training*. What does that cost? Same forest, same 29 deployed features, same board:

Engine	Throughput (flows/s)	p95 (ms)	RAM (%)	Energy/inf. (mJ)
PySpark (deployed)	22.6	532	24.3	71.6
scikit-learn	204.4	49.8	13.3	2.91
ONNX Runtime	**7870.8**	**1.1**	15.7	**0.06**

ONNX Runtime sustains 348× the deployed PySpark throughput. We report this as a *target for the next iteration* — export to ONNX/TensorRT for serving, keep Spark for training.

Conclusion

What we showed

- A reproducible distributed edge IDS: Kafka + PySpark on a Jetson Orin Nano Super cluster.
- Splitting gate and classifier across boards: **2.6 → 62.5 flows/s** at p95 ≈13 ms, gate skipping 95.8%.
- Energy per verdict: **2.5 J → 0.18 J**.

Limitations

- Two-node lab cluster over Wi-Fi; CICIDS2017 replay, not live traffic.
- The ≈67-TOPS GPU is unused by the CPU-only pipeline.
- Evasive traffic untested.

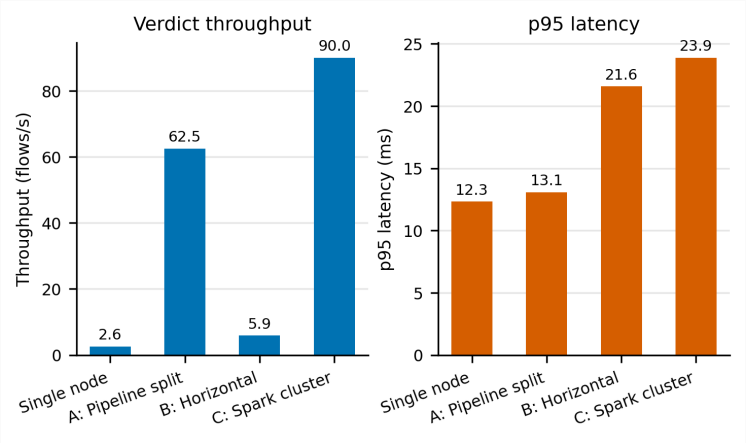
Next

- TensorRT/ONNX in the streaming path.
- Live traffic; federated retraining under drift.

Thank you — questions?

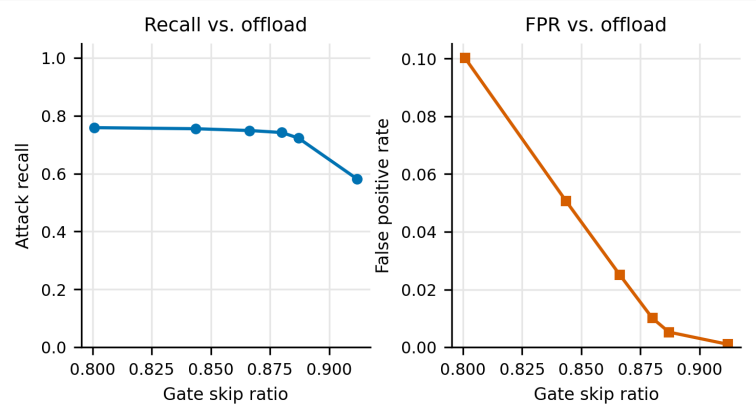
Backup slides

Backup: deployment-mode comparison chart



Verdict throughput (left) and p95 latency (right).

Backup: the gate is an operating curve, not a point



Attack recall (left) and false-positive rate (right) vs. gate-skip ratio. Raising the skip ratio offloads more Spark work and costs recall; the operator chooses the point.

Backup: which model, and why

Configuration	F1	Features	Size (MB)	Edge rationale
Baseline + XGBoost	0.9971	77	2.32	Reference accuracy
SHAP Top-30 + XGBoost	0.9970	30	0.003†	Explainable; deployed set
RF Top-30 + XGBoost	0.9922	30	2.52	Embedded ranking
PCA k=40 + XGBoost	0.9939	40	3.82	Unsupervised reduction

We deploy **SHAP Top-30 with Random Forest** (offline F1 0.9955): a native Spark ML estimator, so the exported `PipelineModel` loads on ARM64 without the native libraries Spark's XGBoost integration needs. The input vector has **29** features — `destination_port` is in the SHAP ranking but excluded as label-leaking.

†Distributed saving under-reports size; true size ≈2.5 MB.

Backup: full monitoring dashboard

Per-node throughput, detected attacks, inference latency, CPU/RAM for both boards, PostgreSQL-backed alerts.

Backup: measurement protocol

- Host-side orchestration starts pipeline roles on the Jetsons over SSH, drives the load, collects per-node logs.
- $\geq$3 repetitions per configuration; warmup traffic *excluded* from the measured window (no JVM/JIT inflation of early batches).
- Metric queries aligned to the exact load window, not a trailing range.
- Latency percentiles per node over *raw per-flow* samples; cluster p95 is conservatively the *worst node's* p95.
- End-to-end send-to-verdict latency from producer timestamps under NTP-synchronised clocks (excludes DB-write cost).
- Energy: `tegrastats`, 30 s idle baseline then load window; two-node modes sum both boards.